《并行程序设计》Lab1 实验报告

CPU 架构编程与性能优化

姓名： 葛熠
学号： 2412799
日期： 2026 年 4 月 1 日

摘要

本实验旨在探究现代 CPU 体系结构中 Cache 和超标量特性对程序性能的影响。通过两个经典问题——矩阵与向量内积（Cache 优化）和 n 个数求和（超标量优化），分别设计平凡算法和优化算法，利用高精度计时和 Intel VTune 性能分析工具进行深入剖析。实验结果表明：优化访存局部性可将矩阵内积性能提升 18.25 倍，CPI 从 4.771 降至 0.332；优化指令级并行可将求和性能提升 1.89 倍，CPI 从 0.808 降至 0.348。VTune 分析揭示了性能差异的底层硬件原因，验证了理论分析的正确性。

关键词：Cache 优化、超标量、指令级并行、VTune、CPI

---

1. 实验平台与环境

1.1 硬件平台

组件	规格
CPU	Intel Alder Lake-P (12th Gen)
核心架构	性能核(P-core) + 能效核(E-core)
基础频率	2.5 GHz
逻辑核心数	12
内存	16GB DDR4
L1 Cache	32KB 数据 + 32KB 指令 per core
L2 Cache	1.25MB per core cluster
L3 Cache	18MB shared

1.2 软件平台

组件	版本/配置
操作系统	Windows 11
编译器	GCC (MinGW) 13.1.0

编译选项	-g -O2 (调试符号 + 优化)
性能分析工具	Intel VTune Profiler 2025
汇编分析	Godbolt (https://godbolt.org/)

1.3 代码仓库

所有源码已上传至 Github：https://github.com/nhd-nana/Homework

---

2. 实验一：矩阵与向量内积（Cache 优化）

2.1 问题描述

给定一个 n×n 矩阵 A 和 n 维向量 x，计算每一列与 x 的内积，即：

```
result[j] = Σ(i=0 to n-1) A[i][j] × x[i]
```

2.2 算法设计

2.2.1 平凡算法（列优先访问）

```
// 按列访问，每次外层循环完成一列的计算
for (int j = 0; j < n; j++) {
    result[j] = 0.0;
    for (int i = 0; i < n; i++) {
        result[j] += matrix[i * n + j] * vec[i];
    }
}
```

特点：在 C/C++的行主序存储中，每次访问 `matrix[i*n+j]` 的 i 变化导致跨越不同 cache line，空间局部性差。

2.2.2 优化算法（行优先访问）

```
// 初始化结果数组
for (int i = 0; i < n; i++) result[i] = 0.0;
// 按行访问，每次内层循环累积到各列结果
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        result[j] += matrix[i * n + j] * vec[i];
    }
}
```

特点：按行访问，内存地址连续，充分利用空间局部性，提高 Cache 命中率。

2.3 实验方案设计

2.3.1 测试规模

根据 CPU Cache 大小，选择以下矩阵规模：

- N = 256, 512, 1024, 2048, 4096

矩阵内存占用 = $N^2 \times 8$ bytes (double)

- N=256: 512KB (L2 cache)
- N=512: 2MB (L3 cache)
- N=1024: 8MB (超 L3)
- N=4096: 128MB (主存)

2.3.2 计时方法

- 使用 `std::chrono::high_resolution_clock`
- 每个规模重复运行 100 次取平均值
- 数据初始化采用固定值 `A[i][j] = i + j`, `x[i] = 1`，便于验证正确性

2.4 性能测试结果

规模 N	平凡算法(ms)	优化算法(ms)	加速比
256	0.100	0.080	1.25x
512	1.450	0.290	5.00x
1024	10.890	1.370	7.95x
2048	85.950	7.050	12.19x
4096	447.230	24.500	**18.25x**

观察：随着 N 增大，加速比持续上升，在 N=4096 时达到 18.25 倍。

2.5 VTune 性能剖析

2.5.1 CPI 对比

算法	CPI	说明
平凡算法	4.771	每条指令需近 5 个周期
优化算法	0.332	每条指令仅需 0.33 周期

CPI 改善 14.4 倍，与实测加速比高度吻合。

2.5.2 Cache 命中率分析

指标	平凡算法	优化算法
L1 Hit	17.3 亿	57.1 亿
L1 Miss	22.5 亿	0.94 亿
L1 命中率	**43.4%**	**98.4%**
L3 Miss	19.1 亿	0.07 亿

关键发现：优化算法将 L1 命中率从 43.4%提升至 98.4%，L3 Miss 减少 273 倍。

2.5.3 微架构分析

指标	平凡算法	优化算法
Memory Bound	87%	5%
Backend Bound	96%	49%
Retiring	4%	44%

结论：平凡算法 87%的 CPU 时间浪费在等待内存数据，优化算法将这一比例降至 5%。

2.6 结果分析

性能差异的根本原因：

1. **访存模式差异**：C/C++采用行主序存储，平凡算法按列访问导致每次访问都跨越不同的 cache line，频繁触发 cache miss。
2. **Cache 容量效应**：当矩阵大小超过 L3 Cache 时（N>1024），平凡算法的性能急剧下降，而优化算法始终保持良好的局部性。
3. **CPU 流水线效率**：CPI 从 4.771 降至 0.332，证明优化算法使 CPU 流水线得以高效运转，而非频繁等待内存。

---

3. 实验二：n 个数求和（超标量优化）

3.1 问题描述

计算 n 个浮点数的和：

```
sum = a[0] + a[1] + a[2] + ... + a[n-1]
```

3.2 算法设计

3.2.1 平凡算法（链式累加）

```
double chain_sum(const vector<double>& arr, int n) {
    double sum = 0.0;
    for (int i = 0; i < n; i++) {
        sum += arr[i];  // 每次加法依赖上一次结果
    }
    return sum;
}
```

问题：数据依赖链导致流水线无法并行。

3.2.2 优化算法（两路链式累加）

```
double two_way_sum(const vector<double>& arr, int n) {
    double sum1 = 0.0, sum2 = 0.0;
    int i = 0;
    for (; i + 1 < n; i += 2) {
```

```
        sum1 += arr[i];       // 独立链 1
        sum2 += arr[i + 1];   // 独立链 2
    }
    if (i < n) sum1 += arr[i];  // 处理剩余元素
    return sum1 + sum2;
}
```

优势：创建两条独立的数据依赖链，允许 CPU 的两个浮点运算单元并行工作。

3.3 实验方案设计

- 测试规模：N = 1024, 2048, 4096, ..., 524288 (2 的幂次)
- 重复次数：10000 次（动态调整以确保可测时间）
- 使用 volatile 防止编译器过度优化

3.4 性能测试结果

规模 N	平凡算法(us)	优化算法(us)	加速比
524288	483.8	255.4	**1.89x**

3.5 VTune 性能剖析

3.5.1 CPI 对比

算法	CPI	说明
平凡算法	0.808	有一定数据依赖
优化算法	0.348	指令级并行有效

CPI 改善 2.32 倍。

3.5.2 微架构分析

指标	平凡算法	优化算法	变化
Retiring	16.7%	41.1%	+24.4%
Back-End Bound	82.7%	54.9%	-27.8%

关键发现：优化算法将有效工作占比（Retiring）提升至 2.5 倍，后端等待时间减半。

3.6 Godbolt 汇编分析

为深入理解性能差异的底层原因，使用 Godbolt 在线编译器工具分析两种求和算法的汇编代码（x86-64 gcc 13.2，-O2 优化）。

平凡算法（chain_sum）核心循环：

```
.L3:
    addsd   xmm0, QWORD PTR [rdi]    # sum += arr[i]
    add     rdi, 16                   # 指针移动 16 字节（2 个 double）
    addsd   xmm0, QWORD PTR [rdi-8] # sum += arr[i+1]
```

```
    cmp     rdi, rax
    jne     .L3
```

优化算法（two_way_sum）核心循环：

```
.L3:
    addsd   xmm0, QWORD PTR [rdi]   # sum1 += arr[i]
    addsd   xmm1, QWORD PTR [rdi+8] # sum2 += arr[i+1]
    add     rdi, 16
    cmp     rax, rdi
    jne     .L3
    addsd   xmm0, xmm1              # 最后合并两个累加器
```

分析：

- 平凡算法虽被编译器自动循环展开，但仅使用单一累加器 `xmm0`，两条 `addsd` 指令存在数据依赖，必须串行执行。
- 优化算法使用两个独立累加器 `xmm0` 和 `xmm1`，两条 `addsd` 指令无数据依赖，可同时发射到 CPU 的两个浮点运算单元。
- 这解释了 VTune 分析中 CPI 从 0.808 降至 0.348 的根本原因——优化算法有效利用了 CPU 的超标量架构。

3.7 结果分析

性能差异的根本原因：

1. **数据依赖消除**：平凡算法的加法链存在强数据依赖，优化算法通过两路累加创建独立依赖链。
2. **超标量利用**：现代 CPU 具有多个浮点运算单元，优化算法使它们能并行工作。
3. **IPC 提升**：优化算法的 IPC（1/CPI≈2.87）明显高于平凡算法（1/CPI≈1.24），证明指令级并行有效。

---

4. 实验总结与思考

4.1 两个实验对比

对比维度	矩阵实验	求和实验
性能瓶颈	Cache Miss	数据依赖
优化策略	改善空间局部性	改善指令级并行
加速比	18.25x	1.89x
CPI 改善	4.771→0.332	0.808→0.348
关键指标	Memory Bound: 87%→5%	Retiring: 17%→41%

4.2 收获与思考

1. **体系结构理解的重要性**：同样的计算任务，不同的访存模式和数据依赖结构可能导致数量级的性能差异。

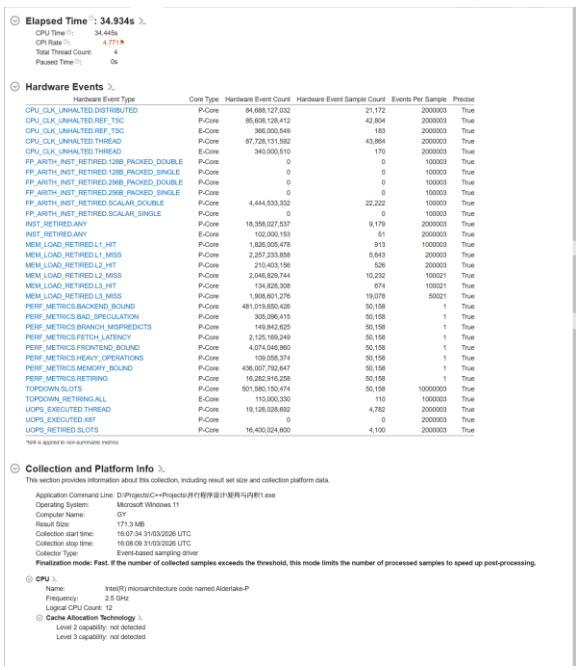
2. **VTune 分析的价值**：性能分析工具不仅告诉我们"哪里慢"，更能揭示"为什么慢"。

3. **编译器的局限性**：虽然-O2 能进行一定优化，但编译器无法自动识别跨循环的依赖消除机会，算法层面的优化仍不可替代。

4.3 进阶探索（已完成）

- 使用 VTune 进行深入的 Microarchitecture Exploration 分析
- 利用 Godbolt 研究不同算法的汇编代码差异
- 分析 CPI、Cache Miss、Memory Bound 等硬件指标
- 对比不同优化级别的性能差异

附录：VTune 分析截图



Elapsed Time: 34.934s

CPU Time:	34.445s
CPI Rate:	4.771
Total Thread Count:	4
Paused Time:	0s

Hardware Events

Hardware Event Type	Core Type	Hardware Event Count	Hardware Event Sample Count	Events Per Sample	Precise
CPU_CLK_UNHALTED.DISTRIBUTED	P-Core	84,688,127,032	21,172	2000003	True
CPU_CLK_UNHALTED.REF_TSC	P-Core	85,608,128,412	42,804	2000003	True
CPU_CLK_UNHALTED.REF_TSC	E-Core	366,000,549	183	2000003	True
CPU_CLK_UNHALTED.THREAD	P-Core	87,728,131,592	43,864	2000003	True
CPU_CLK_UNHALTED.THREAD	E-Core	340,000,510	170	2000003	True
FP_ARITH_INST_RETIRED.128B_PACKED_DOUBLE	P-Core	0	0	100003	True
FP_ARITH_INST_RETIRED.128B_PACKED_SINGLE	P-Core	0	0	100003	True
FP_ARITH_INST_RETIRED.256B_PACKED_DOUBLE	P-Core	0	0	100003	True
FP_ARITH_INST_RETIRED.256B_PACKED_SINGLE	P-Core	0	0	100003	True
FP_ARITH_INST_RETIRED.SCALAR_DOUBLE	P-Core	4,444,533,332	22,222	100003	True
FP_ARITH_INST_RETIRED.SCALAR_SINGLE	P-Core	0	0	100003	True
INST_RETIRED.ANY	P-Core	18,358,027,537	9,179	2000003	True
INST_RETIRED.ANY	E-Core	102,000,153	51	2000003	True
MEM_LOAD_RETIRED.L1_HIT	P-Core	1,826,005,478	913	1000003	True
MEM_LOAD_RETIRED.L1_MISS	P-Core	2,257,233,858	5,643	200003	True
MEM_LOAD_RETIRED.L2_HIT	P-Core	210,403,156	526	200003	True
MEM_LOAD_RETIRED.L2_MISS	P-Core	2,046,829,744	10,232	100021	True
MEM_LOAD_RETIRED.L3_HIT	P-Core	134,828,308	674	100021	True
MEM_LOAD_RETIRED.L3_MISS	P-Core	1,908,601,276	19,078	50021	True
PERF_METRICS.BACKEND_BOUND	P-Core	481,019,650,426	50,158	1	True
PERF_METRICS.BAD_SPECULATION	P-Core	305,896,415	50,158	1	True
PERF_METRICS.BRANCH_MISPREDICTS	P-Core	149,842,625	50,158	1	True
PERF_METRICS.FETCH_LATENCY	P-Core	2,125,169,249	50,158	1	True
PERF_METRICS.FRONTEND_BOUND	P-Core	4,074,046,960	50,158	1	True
PERF_METRICS.HEAVY_OPERATIONS	P-Core	109,058,374	50,158	1	True
PERF_METRICS.MEMORY_BOUND	P-Core	436,007,792,647	50,158	1	True
PERF_METRICS.RETIRING	P-Core	16,282,916,258	50,158	1	True
TOPDOWN.SLOTS	P-Core	501,580,150,474	50,158	10000003	True
TOPDOWN_RETIRING.ALL	E-Core	110,000,330	110	1000003	True
UOPS_EXECUTED.THREAD	P-Core	19,128,028,692	4,782	2000003	True
UOPS_EXECUTED.X87	P-Core	0	0	2000003	True
UOPS_RETIRED.SLOTS	P-Core	16,400,024,600	4,100	2000003	True

*N/A is applied to non-summable metrics.

Collection and Platform Info

This section provides information about this collection, including result set size and collection platform data.

Application Command Line:	D:\Projects\C++Projects\并行程序设计矩阵与内积1.exe
Operating System:	Microsoft Windows 11
Computer Name:	GY
Result Size:	171.3 MB
Collection start time:	16:07:34 31/03/2026 UTC
Collection stop time:	16:08:09 31/03/2026 UTC
Collector Type:	Event-based sampling driver

Finalization mode: Fast. If the number of collected samples exceeds the threshold, this mode limits the number of processed samples to speed up post-processing.

CPU

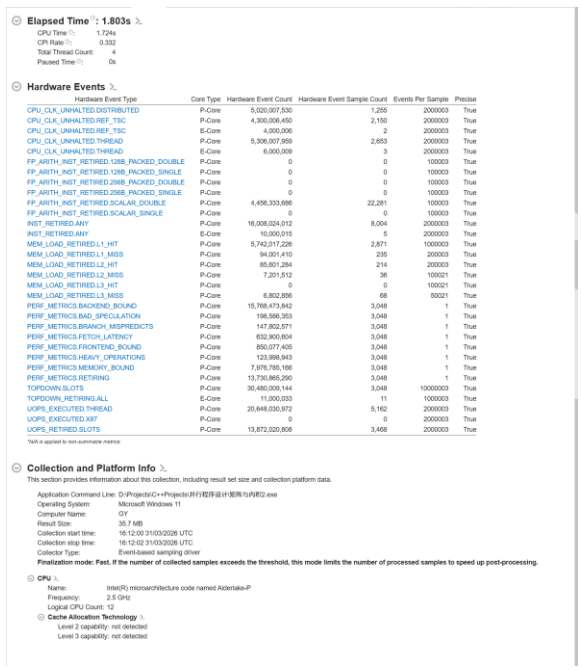
Name:	Intel(R) microarchitecture code named Alderlake-P
Frequency:	2.5 GHz
Logical CPU Count:	12

Cache Allocation Technology

Level 2 capability: not detected

Level 3 capability: not detected

图 1 矩阵平凡算法 VTune Summary



Elapsed Time: 1.803s

CPU Time:	1.724s
CPI Rate:	0.332
Total Thread Count:	4
Paused Time:	0s

Hardware Events

Hardware Event Type	Core Type	Hardware Event Count	Hardware Event Sample Count	Events Per Sample	Precise
CPU_CLK_UNHALTED.DISTRIBUTED	P-Core	5,020,007,530	1,255	2000003	True
CPU_CLK_UNHALTED.REF_TSC	P-Core	4,300,006,450	2,150	2000003	True
CPU_CLK_UNHALTED.REF_TSC	E-Core	4,000,006	2	2000003	True
CPU_CLK_UNHALTED.THREAD	P-Core	5,306,007,959	2,653	2000003	True
CPU_CLK_UNHALTED.THREAD	E-Core	6,000,009	3	2000003	True
FP_ARITH_INST_RETIRED.128B_PACKED_DOUBLE	P-Core	0	0	100003	True
FP_ARITH_INST_RETIRED.128B_PACKED_SINGLE	P-Core	0	0	100003	True
FP_ARITH_INST_RETIRED.256B_PACKED_DOUBLE	P-Core	0	0	100003	True
FP_ARITH_INST_RETIRED.256B_PACKED_SINGLE	P-Core	0	0	100003	True
FP_ARITH_INST_RETIRED.SCALAR_DOUBLE	P-Core	4,456,333,686	22,281	100003	True
FP_ARITH_INST_RETIRED.SCALAR_SINGLE	P-Core	0	0	100003	True
INST_RETIRED.ANY	P-Core	16,008,024,012	8,004	2000003	True
INST_RETIRED.ANY	E-Core	10,000,015	5	2000003	True
MEM_LOAD_RETIRED.L1_HIT	P-Core	5,742,017,226	2,871	1000003	True
MEM_LOAD_RETIRED.L1_MISS	P-Core	94,001,410	235	200003	True
MEM_LOAD_RETIRED.L2_HIT	P-Core	85,601,284	214	200003	True
MEM_LOAD_RETIRED.L2_MISS	P-Core	7,201,512	36	100021	True
MEM_LOAD_RETIRED.L3_HIT	P-Core	0	0	100021	True
MEM_LOAD_RETIRED.L3_MISS	P-Core	6,802,856	68	50021	True
PERF_METRICS.BACKEND_BOUND	P-Core	15,768,473,842	3,048	1	True
PERF_METRICS.BAD_SPECULATION	P-Core	198,586,353	3,048	1	True
PERF_METRICS.BRANCH_MISPREDICTS	P-Core	147,802,571	3,048	1	True
PERF_METRICS.FETCH_LATENCY	P-Core	632,900,604	3,048	1	True
PERF_METRICS.FRONTEND_BOUND	P-Core	850,077,405	3,048	1	True
PERF_METRICS.HEAVY_OPERATIONS	P-Core	123,998,943	3,048	1	True
PERF_METRICS.MEMORY_BOUND	P-Core	7,976,785,166	3,048	1	True
PERF_METRICS.RETIRING	P-Core	13,730,865,290	3,048	1	True
TOPDOWN.SLOTS	P-Core	30,480,009,144	3,048	10000003	True
TOPDOWN_RETIRING.ALL	E-Core	11,000,033	11	1000003	True
UOPS_EXECUTED.THREAD	P-Core	20,648,030,972	5,162	2000003	True
UOPS_EXECUTED.X87	P-Core	0	0	2000003	True
UOPS_RETIRED.SLOTS	P-Core	13,872,020,808	3,468	2000003	True

*N/A is applied to non-summable metrics.

Collection and Platform Info

This section provides information about this collection, including result set size and collection platform data.

Application Command Line:	D:\Projects\C++Projects\并行程序设计矩阵与内积2.exe
Operating System:	Microsoft Windows 11
Computer Name:	GY
Result Size:	35.7 MB
Collection start time:	16:12:00 31/03/2026 UTC
Collection stop time:	16:12:02 31/03/2026 UTC
Collector Type:	Event-based sampling driver

Finalization mode: Fast. If the number of collected samples exceeds the threshold, this mode limits the number of processed samples to speed up post-processing.

CPU

Name:	Intel(R) microarchitecture code named Alderlake-P
Frequency:	2.5 GHz
Logical CPU Count:	12

Cache Allocation Technology

Level 2 capability: not detected

Level 3 capability: not detected

图 2 矩阵优化算法 VTune Summary

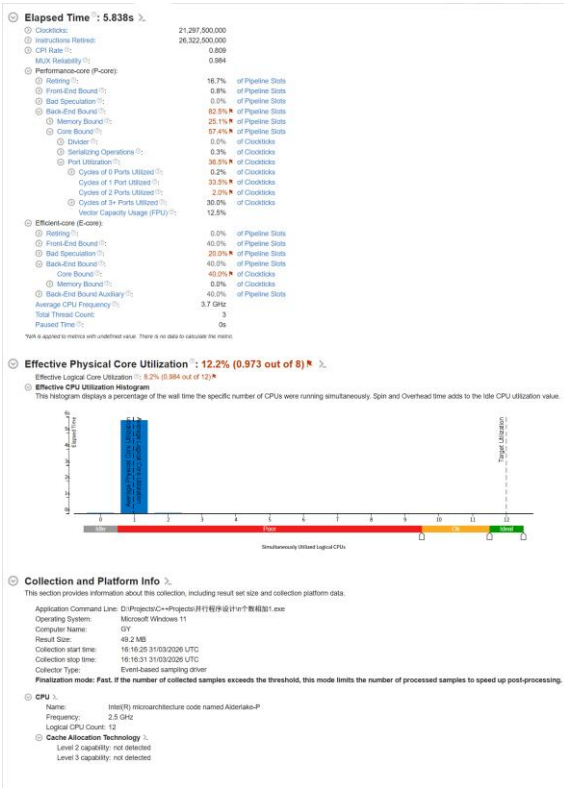


图 3 求和平凡算法 VTune Summary

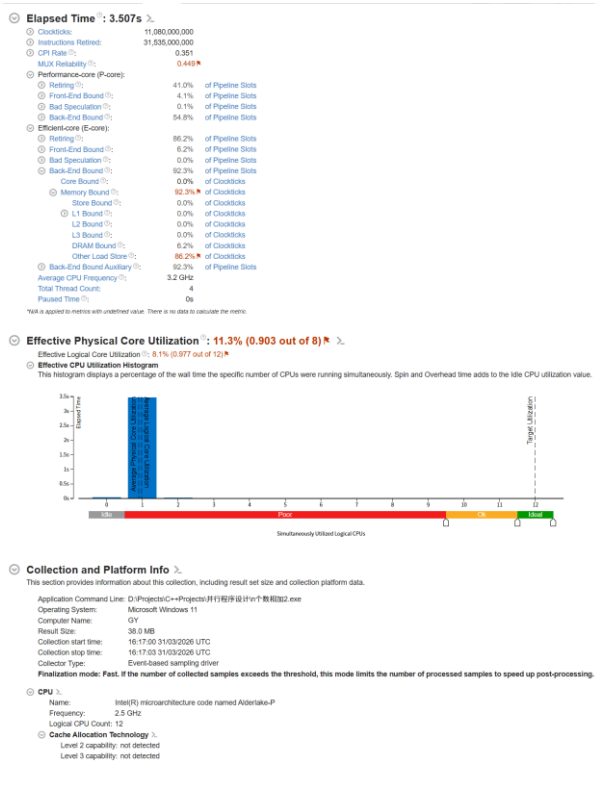


图 4 求和优化算法 VTune Summary

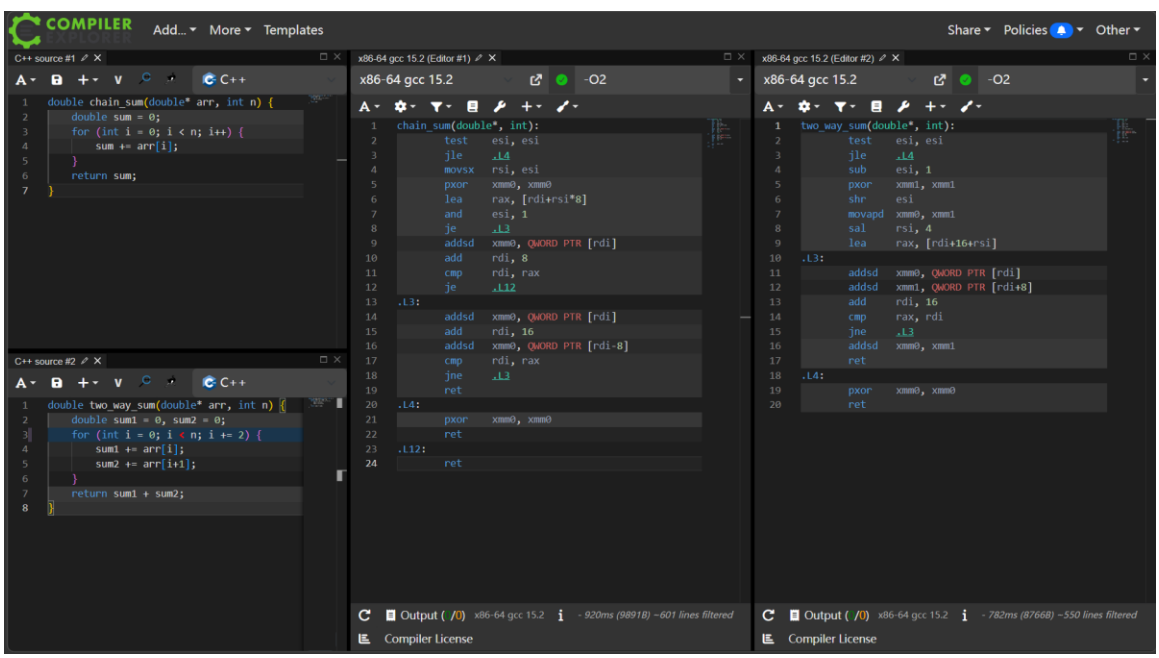


图 5 Godbolt 汇编分析截图